

5. Project Development Phase

5.1 Core Backend Controller & Type Boundaries

The system relies on strict Pydantic parsing types to validate incoming data structures before running analytical processing loops. The code snippet below details the production controller handling asynchronous payload sanitization:

```
from fastapi import FastAPI, Depends, HTTPException, status
from pydantic import BaseModel, Field
from uuid import UUID
from datetime import datetime
import json

app = FastAPI(title="Personalized Networking Assistant Core Engine")

class InteractionLogSchema(BaseModel):
    contact_id: UUID = Field(..., description="Target reference profile identity")
    interaction_type: str = Field(..., regex="^(Email|Meeting|Call|Conference)$")
    interaction_date: datetime = Field(default_factory=datetime.utcnow)
    conversation_summary: str = Field(..., min_length=10, max_length=4096)
    intensity_score: int = Field(..., ge=1, le=5)

@app.post("/api/v1/interactions/log", status_code=status.HTTP_201_CREATED)
async def register_new_interaction(payload: InteractionLogSchema):
    try:
        # Step 1: Validate payload schema bounds using defined structures
        cleaned_data = payload.dict()

        # Step 2: Bind parsed information to database persistence context
        status_msg = f"Interaction profile recorded for contact reference {cleaned_data['contact_id']}"

        # Step 3: Trigger asynchronous background update jobs
        # celery_worker.send_task("compute_decay_delta", args=[cleaned_data['contact_id']])

        return {
            "status": "success",
            "message": status_msg,
            "timestamp": datetime.utcnow()
        }
    except Exception as exc:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Internal database transaction mapping exception: {str(exc)}"
        )
```

This data layer filters and limits textual inputs, ensuring clean strings are passed to downstream analytical components or vector embedding services.

5.2 RESTful Ingestion APIs & Services

The backend engine exposes structured endpoints for external connections and frontend interfaces:

- **GET /api/v1/contacts/metrics**
Fetches user connection states sorted by urgency, listing computed degradation scores and days since last contact.
- **POST /api/v1/ai/generate-context**
Analyzes historical meeting text logs to generate customized, context-aware interaction templates.